GORGO: Online Tuning for Cross-Region Network-Aware LLM Serving

Anonymous Author(s)

Abstract

In cross-region LLM serving, time-to-first-token (TTFT) is highly sensitive to per-request routing decisions. KV-cache reuse across replicas can eliminate much of the prefill computation on multi-turn workloads, but realizing this potential requires jointly balancing cache locality against replica load and network cost. Additionally, public datasets to evaluate these policies do not represent long-context, high prefix-reuse workloads: LMSYS and WildChat average 2 and 2.5 turns per conversation, 0.5 and 2.9k tokens/request, and 5 and 9% intra-user reuse, respectively. To capture the setting where a majority of requests contain reusable prefixes, we evaluate common routing policies such as least-load, prefix-aware, session-affinity, and others on traces from a production inference endpoint featuring multi-turn conversations ($\sim$82 requests/user), 7–45$\times$ longer prompts (21k avg tokens), and 2–6$\times$ higher KV-cache reuse (55%). We evaluate policies on a multi-region GPU cluster where cross-region network latency ranges from 26ms to 466ms across replicas to test GORGO, a policy that jointly optimizes cache reuse, network latency, and queue depth with weights optimized online to minimize p95 TTFT with no offline profiling. The GORGO family collectively achieves the lowest TTFT at every percentile (p50, p95, p99) across both a tuning window and a held-out evaluation window against all other load balancing policies, highlighting the impact of online joint optimization.

1 Introduction

Modern LLM chat services serve interactive workloads where perceived latency is dominated by time-to-first-token (TTFT). On a single replica, TTFT is the sum of three terms: (i) the network round-trip from the client proxy to the replica, (ii) queueing delay behind in-flight requests, and (iii) prefill computation for the input prompt. Inference engines such as SGLang [Zheng et al., 2024b] and vLLM [Kwon et al., 2023] expose radix-tree prefix caches that let an incoming request skip prefill for any token prefix already cached on the handling replica. When prompts are long and prefixes are widely reused, this saving dominates: a 20,000-token prompt that hits a 90% prefix match has only $\sim$2,000 tokens to prefill, reducing TTFT by an order of magnitude.

The KV-cache reuse opportunity is contingent on routing. If a request is sent to a replica that has not seen its prefix, the saving is forfeit. If many requests pile onto the same warm replica, that replica becomes a queueing bottleneck and the cache hit no longer pays for itself. If the closest replica with a warm cache is on the other side of the world, the network round-trip can swallow what the cache hit was supposed to save. The routing layer therefore has to balance three terms simultaneously: network distance to the replica, current load, and reusable cache state for the incoming prompt.

Standard heuristics—least-request, least-load, prefix-aware routing, session-affinity—each optimize one of these signals and ignore the others. For workloads with weak prefix reuse and uniform replica latencies, these heuristics are standard practice. Interactive long-context traffic breaks both assumptions: production chat traces concentrate most tokens in a small set of returning users with substantial prefix overlap, and replicas placed in different regions have round-trip times that differ

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

by an order of magnitude. The right routing target for a given request can change minute-to-minute as load shifts, and weighting the three terms with static rules leaves performance on the table.

Two further factors complicate empirical evaluation. First, the public chat datasets used in prior LLM-serving evaluations underrepresent long-context multi-turn traffic with reusable prefixes. LMSYS-Chat-1M [Zheng et al., 2024a] and WildChat-4.8M [Zhao et al., 2024] average 470 and 2,900 tokens per request, with intra-user prefix reuse below 10%. A production endpoint we evaluate carries 21,000 tokens per request on average and 53.7% intra-user prefix reuse—a regime where joint cache–network–queue trade-offs dominate TTFT. Second, evaluation methodology matters: a routing policy that wins p50 by herding traffic onto a single warm replica typically loses tail latency once that replica saturates. Reporting only one percentile hides this trade-off.

Contributions.

- A workload characterization (§2) placing three datasets on the same prefix-reuse and request-length axes. The production GLM-5.1 trace has 7–45$\times$ longer prompts and 2–6$\times$ higher token-weighted reuse than LMSYS-Chat-1M and WildChat-4.8M.
- A routing cost model and three variants collectively called GORGO (§3), which score each replica by an additive combination of EWMA network RTT, estimated prefill cost over uncached tokens, and a queue-depth penalty. Variants differ only in how the two scalar weights are set: a static configuration, a per-replica online fit, and a Gaussian $(1{+}1)$-evolution-strategy hill climb that directly minimizes p95 TTFT.
- An experimental setup (§4) running each of eight policies on its own dedicated three-replica fleet of L40S:2 SGLang servers in Asia, Europe, and the US East coast, with cross-region RTTs spanning 26–466 ms.
- Empirical results (§5) on two consecutive 30-minute production-trace windows. GORGO achieves the lowest TTFT at every percentile (p50, p95, p99) on both a tuning and a held-out evaluation window, against seven baseline policies.

We do not claim any single GORGO variant dominates everywhere: `gorgo-hillclimb` wins all three percentiles in the tuning window, while in the evaluation window `gorgo-static` (deploying the tuning-window learned weights) wins p50 and p95 and `gorgo-hillclimb` wins p99. The W2 p99 margin is at the noise floor of one 30-minute window (§6).

2 Workload Characterization

Routing policies that perform well on short, low-reuse prompts may perform differently on long, high-reuse prompts because the prefill term in TTFT scales with uncached input tokens. We measure three datasets along the same axes: request length, conversational structure, and token-weighted prefix reuse.

Datasets. **LMSYS-Chat-1M** [Zheng et al., 2024a] is the Chatbot Arena dataset of one million conversations. **WildChat-4.8M** [Zhao et al., 2024] is a corpus of 3.2M ChatGPT-proxy conversations grouped by client IP. **GLM-5.1 production trace** is a 7-day chat-completion log from a long-context production endpoint [GLM Team, 2024]. It contains 411,169 requests across 4,984 distinct users, yielding $\sim$82 requests per user on average.

We measure prefix reuse with a token-weighted prefix trie that ingests each request's tokenized message sequence in arrival order, partitioned by user where the dataset carries user identity. The trie reports *intra-user savings* (fraction of tokens matching a prefix from the same user's prior requests), *cross-user savings* (additional fraction from pooling across users), and *global savings* (sum). LMSYS does not carry user identity, so we report only the global figure.

Three contrasts stand out (Table 1). *Request length*: GLM-5.1 prompts are 45$\times$ longer than LMSYS and 7$\times$ longer than WildChat. At typical prefill rates, a 21,000-token uncached prompt takes seconds. *User concentration*: GLM-5.1 has 82 requests per user on average; WildChat has 1.7; LMSYS has effectively one. Intra-user reuse depends on returning users, which the public sets simply do not have. *Reuse magnitude*: 53.7% of GLM-5.1 tokens are part of a prefix seen earlier from the same user (5.3% for WildChat, essentially zero for LMSYS). The bulk of WildChat's 34.4% global

Table 1: Workload characterization. Token counts use the same byte-pair tokenizer in all rows. Reuse percentages are token-weighted prefix-trie savings as described in §2.

Dataset	Total reqs	Users	Avg input tokens	Intra-user reuse	Global reuse
LMSYS-Chat-1M	1,000,000	—	467	—	9.0%
WildChat-4.8M	3,199,860	1,833,730	2,925	5.3%	34.4%
GLM-5.1 production	411,169	4,984	21,043	53.7%	55.3%

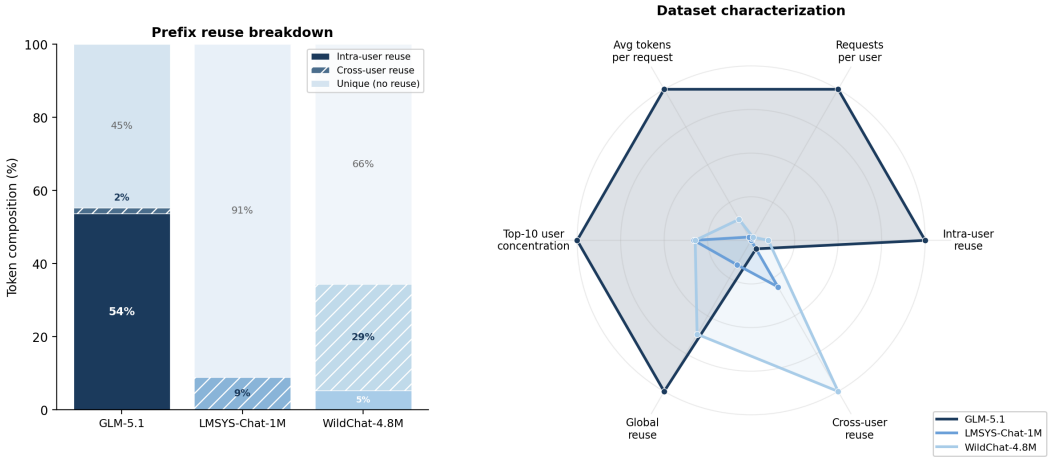

Figure 1: *Left*: Token composition per dataset. GLM-5.1's reuse is overwhelmingly intra-user (54%), reflecting multi-turn dialogue; WildChat's is predominantly cross-user (29%, shared templates); LMSYS has minimal reuse (9%, all cross-user). *Right*: Radar chart with six axes normalized to the dataset maximum. GLM-5.1 dominates on every axis relevant to routing: long prompts, high multi-turn density, concentrated users, and strong intra-user prefix reuse.

reuse comes from cross-user shared structure (chat templates, viral prompts), not sustained per-user dialogue.

For routing, LMSYS and WildChat live in a regime where prefix-aware policies have little to win. On GLM-5.1, a strategy that lands a user on a replica that has seen their conversation can save more than half of the prefill.

Experimental windows. We focus on two consecutive 30-minute windows of GLM-5.1 traffic: 00:30–01:00 UTC (W1, tuning) and 01:00–01:30 UTC (W2, held-out evaluation). Traces are replayed at original arrival speed and pre-filtered to 24,000 input tokens. Each window contains approximately 4,800 (W1) and 4,200 (W2) requests. Output is capped at 128 tokens so TTFT, not decode, dominates the latency budget.

3 GORGO

3.1 TTFT Decomposition

Let a request r with token sequence x_r be routed to replica u. Its TTFT decomposes as

$$\mathrm{TTFT}(r,u) = \mathrm{rtt}(u) + w_{\mathrm{queue}}(u,t_r) + t_{\mathrm{prefill}}(u) \cdot |x_r \setminus c_u(t_r)| + \varepsilon, \tag{1}$$

where $\mathrm{rtt}(u)$ is the round-trip from the routing proxy to u; $w_{\mathrm{queue}}(u,t_r)$ is queueing delay; $t_{\mathrm{prefill}}(u)$ is the per-token prefill rate; $c_u(t_r)$ is the set of token sequences cached on u at arrival time t_r; and ε collects scheduler overhead and batching variance. Cross-region $\mathrm{rtt}(u)$ ranges from 26 ms to 466 ms in our cluster; the prefill term can be the largest and most routing-sensitive term at typical rates of 0.1–1 ms per uncached token on a 21,000-token prompt.

3.2 Cost Model

A GORGO replica score is a weighted sum of the three controllable terms in Equation 1:

$$\text{score}(u, r) = \text{rtt}_u + t_{\text{prefill}} \cdot (|x_r| - |x_r \cap c_u|) + \alpha \cdot (\text{queued\_tokens}_u + \text{used\_kv\_tokens}_u). \quad (2)$$

The proxy sends r to the replica with the minimum score. Three signals feed the score: (i) rtt_u from the EWMA of a lightweight network probe decoupled from the metrics scrape; (ii) $|x_r \cap c_u|$ from the proxy's local radix trie of cached prefixes per replica, refreshed on every request completion; and (iii) load from the proxy's in-flight token counter combined with SGLang's reported KV-cache occupancy. The two free parameters are the per-token prefill rate t_{prefill} and the load weight α.

3.3 Three Variants of Weight Selection

gorgo-static fixes t_{prefill} and α before the run and never adjusts them. In W1 we use manually chosen starter values ($\alpha = 0.06$, $t_{\text{prefill}} = 0.07$). In W2, `gorgo-static` deploys the per-replica weights that `gorgo-hillclimb` ended W1 at ($t_{\text{prefill}} \approx 0.983$, $\alpha \approx 4.4 \times 10^{-4}$) and never adjusts them again. This isolates the contribution of the cost model independent of online adaptation.

gorgo-autotune re-fits both parameters every 16 new request samples by taking the median of the observed rates $(\text{TTFT} - \text{rtt})/\text{uncached\_tokens}$ from the trailing 64-sample window, partitioned by destination replica. It adapts to per-replica hardware drift and RTT shifts, but treats the cost-model weights as identifiable physical rates rather than black-box knobs.

gorgo-hillclimb runs a Gaussian $(1{+}1)$-evolution strategy [Rechenberg, 1973] over the same two parameters in log-space, with Rechenberg's 1/5-success rule controlling step size. The objective is the negative p95 TTFT of the rolling 64-request window. Every 16 new samples the tuner scores the current weights, accepts or rejects the previous candidate against the incumbent, and proposes a new candidate by perturbing each parameter as $\exp(\log(v) + \sigma\,\mathcal{N}(0, 1))$ with σ adapted according to the recent acceptance rate. This variant treats the cost-model weights as black-box knobs: it finds whatever values produce the best p95 TTFT under the current arrival distribution, with no offline profiling.

4 Experimental Setup

Hardware and software. Each policy runs on its own dedicated three-replica fleet so routing decisions cannot warm or cool another policy's caches. Each fleet consists of one L40S:2 SGLang [Zheng et al., 2024b] server (tensor-parallel 2, 96 GB VRAM per replica) in each of Asia (Seoul), Europe (Frankfurt), and US East (Ashburn), serving Qwen3.5-35B-A3B-FP8 [Qwen Team, 2025]. The model's native context window is 32,768 tokens; we cap workload input at 24,000 tokens to leave KV headroom for in-flight batches. A CPU proxy in US East hosts the routing policy and replays the trace.

The proxy scrapes Prometheus metrics from each replica every $\sim$10 s and runs a separate lightweight HTTP probe to estimate per-replica RTT, smoothed with an EWMA. The probe is decoupled from the metrics scrape to avoid polluting the RTT estimate with SGLang handler contention. Before each run, a homogeneity check issues 16 streaming requests directly to each replica, bypassing the proxy; runs with worst-to-median TTFT ratio exceeding $3\times$ are flagged. No such run occurred in these experiments.

Cross-region RTT distribution. Probed from the US East proxy, EWMA-smoothed RTT to the US East replica is 26–40 ms, to Frankfurt is 90–130 ms, and to Seoul is 260–466 ms. The Seoul upper bound reflects routing-table churn during the trace. The overall range of 26–466 ms represents an $18\times$ spread. Figure 2 shows the full RTT time series over the W1 window.

Baselines, policies, and windows. We compare the three GORGO variants of §3 (`gorgo-static`, `gorgo-autotune`, `gorgo-hillclimb`) against five baseline policies. `random` samples a replica uniformly per request and serves as the lower bound. `least-request` routes to the replica with the fewest in-flight requests, taking the larger of the proxy's in-flight view and SGLang's scraped running-request count to bridge the $\sim$10 s staleness between metrics scrapes. `least-load` routes to the replica with the lowest aggregate token-weighted load, summing running and queued

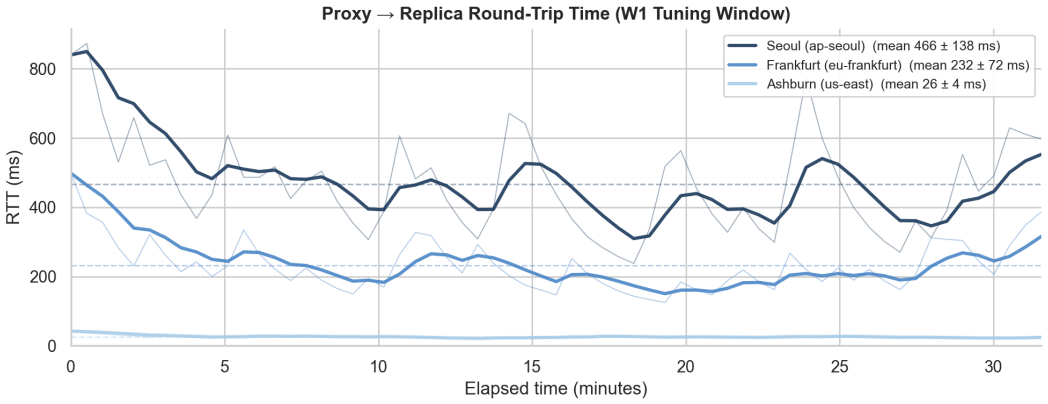

Figure 2: Proxy-to-replica RTT over the W1 tuning window. Bold lines show the EWMA-smoothed signal (α=0.3) that GORGO uses for routing; faint lines show raw probe samples. Dashed lines show per-replica means. Seoul exhibits periodic spikes from routing-table churn; Frankfurt is moderately variable; Ashburn is near-constant. The 18× spread motivates network-aware routing.

Table 2: GLM-5.1 W1 (tuning window, 00:30–01:00 UTC). TTFT in seconds. Bold is best in column. Completed counts from proxy logs; success rate ≈84.2–84.6%.

Policy	Completed	p50	p95	p99	max	Succ.%
gorgo-hillclimb	4,065	**0.288**	**1.466**	**2.307**	4.39	84.4
simple-session-affinity	4,054	0.329	1.482	2.804	14.96	84.2
gorgo-static	4,076	0.299	1.605	2.357	4.16	84.6
least-request	4,074	0.293	1.632	2.355	13.71	84.6
prefix-cache	4,075	0.297	1.654	3.017	5.96	84.6
gorgo-autotune	4,076	0.294	1.658	2.428	4.18	84.6
least-load	4,066	0.297	1.728	2.629	4.64	84.4
random	4,076	0.296	1.828	3.171	28.99	84.6

requests with used and queued KV tokens; it is heavier-signal than least-request but cache-blind. `prefix-cache` is the longest-prefix-match policy of Preble [Srivatsa et al., 2025] as packaged in AIBrix [AIBrix Team, 2024]: it picks the replica with the longest cached prefix match and falls back to least-request when the running-request imbalance exceeds a soft threshold. `simple-session-affinity` hashes the first 256 token IDs of the prompt modulo the number of replicas, maximizing intra-user reuse but providing no load-escape mechanism. We evaluate these eight policies in each of two windows; all eight fleets start simultaneously at the same wall-clock time with the same input trace. W1 seeds the GORGO variants with manually chosen starting weights ($\alpha = 0.06$, $t_{\text{prefill}} = 0.07$); W2 seeds them with the per-replica weights that `gorgo-hillclimb` learned during W1 ($t_{\text{prefill}} \approx 0.983$, $\alpha \approx 4.4 \times 10^{-4}$). `gorgo-static` in W2 deploys those weights without further adjustment; the other GORGO variants continue tuning on the held-out window. Concurrency is 32 in-flight requests per proxy.

Metrics. For each policy and window we report TTFT p50, p95, and p99 over all successful streaming responses. Failed requests (HTTP 4xx/5xx, dominated by prompt-too-long responses at the model's effective context boundary) are excluded from latency percentiles but counted in the per-policy success rate. Success rate is between 84.2% and 85.1% across all policies and windows.

5 Results

5.1 W1: Tuning Window

In W1 (Table 2), `gorgo-hillclimb` delivers the lowest p50 (0.288 s), p95 (1.466 s), and p99 (2.307 s). The next-best p95 is `simple-session-affinity` at 1.482 s, a 1.1% gap. The `gorgo-static` configuration—with manually-set starting weights, never adjusted—already beats

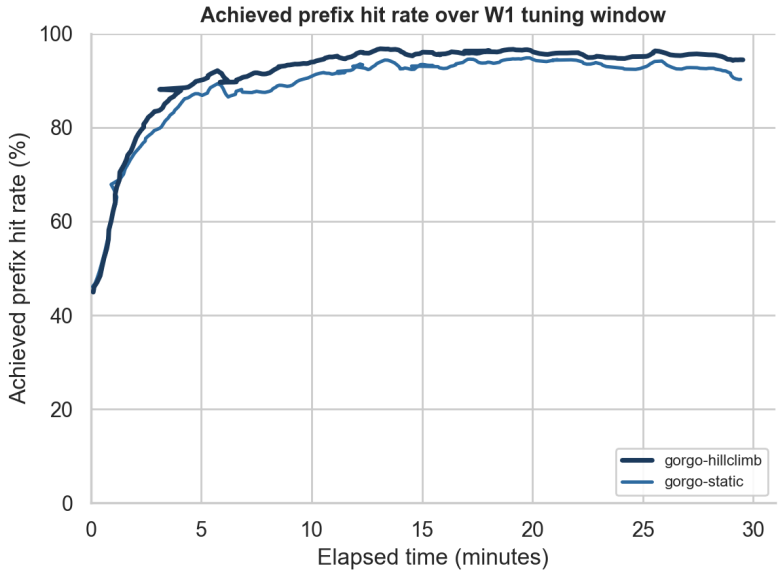

Figure 3: Achieved prefix hit rate over the W1 tuning window: the fraction of each request's input tokens that were already cached on the replica the policy routed to (not the total cache available across all replicas). A higher rate means the routing decision exploited more of the available cache. Bold lines are EWMA-smoothed (α=0.06); faint lines are 64-request rolling averages. `gorgo-hillclimb` converges from $\sim$45% to $\sim$95% within the first 5 minutes as the ES learns to route requests to their best-cached replica.

Table 3: GLM-5.1 W2 (held-out evaluation window, 01:00–01:30 UTC). `gorgo-static` deploys the W1-learned weights and does not adjust them. Bold is best in column.

Policy	Completed	p50	p95	p99	max	Succ.%
gorgo-static	3,573	**0.294**	**1.510**	2.791	15.26	84.9
gorgo-hillclimb	3,583	0.334	1.535	**2.595**	12.68	85.1
prefix-cache	3,578	0.316	1.632	2.860	34.34	85.0
least-request	3,579	0.303	1.757	2.663	5.22	85.1
random	3,575	0.336	1.763	2.612	30.93	85.0
least-load	3,580	0.363	1.813	2.659	3.84	85.1
gorgo-autotune	3,578	0.368	1.888	3.001	5.70	85.0
simple-session-affinity	3,570	0.396	2.080	4.917	36.60	84.8

every non-GORGO policy on p99 and ranks third on p95, showing that the cost model alone improves on single-signal heuristics. The p95 spread across all policies in W1 is 1.25$\times$ (1.466 s vs. 1.828 s).

`prefix-cache` is competitive at p95 (1.654 s) but tail-degraded at p99 (3.017 s) because it concentrates traffic onto warm-cache replicas and pays a queueing penalty when that replica is momentarily saturated. `simple-session-affinity` achieves the second-best p95 on W1 because the production trace has strong per-user temporal locality, but its p99 (2.804 s) and max (14.96 s) reflect episodes where the hashed replica is transiently overloaded with no escape mechanism.

Starting from $\alpha = 0.06$, $t_{\text{prefill}} = 0.07$, the ES converged to $t_{\text{prefill}} \approx 0.983$ and $\alpha \approx 4.4 \times 10^{-4}$ over the 30-minute window. The high learned t_{prefill} reflects the long uncached prefixes in this trace (21k-token prompts, 55% reuse leaves $\sim$9.5k tokens to prefill on average); the low α reflects that SGLang's continuous batching already orders within a replica and deep queues are rare at 32-concurrency on this arrival rate.

Table 4: GLM-5.1 stress, nighttime window (c=64). Same time-of-day band as W1, the day after W1. Bold is best in column. `simple-session-affinity` did not complete and is omitted.

Policy	Completed	p50	p95	p99	max	Succ.%
gorgo-hillclimb	2,466	0.281	**1.331**	**1.832**	3.05	89.5
prefix-cache	2,468	0.288	1.394	1.992	2.95	89.6
gorgo-static	2,472	**0.179**	1.427	2.302	4.06	89.8
least-request	2,465	0.281	1.605	2.268	4.16	89.5
gorgo-autotune	2,469	0.280	1.690	2.298	18.84	89.7
least-load	2,465	0.281	1.700	2.290	2.98	89.5
random	2,460	0.287	1.724	2.357	4.00	89.3

Table 5: GLM-5.1 stress, daytime window (c=64, same day as W1, midday traffic regime). Bold is best in column.

Policy	Completed	p50	p95	p99	max	Succ.%
gorgo-static	3,372	**0.193**	**1.400**	2.140	3.37	89.6
prefix-cache	3,376	0.295	1.416	**1.973**	3.75	89.7
simple-session-affinity	3,375	0.324	1.531	2.487	28.07	89.7
gorgo-hillclimb	3,376	0.226	1.581	7.186	38.29	89.7
least-request	3,380	0.275	1.694	2.440	20.07	89.8
random	3,371	0.309	1.700	2.421	3.99	89.6
least-load	3,374	0.309	1.735	2.419	16.91	89.7
gorgo-autotune	3,377	0.299	1.748	2.519	23.63	89.7

5.2 W2: Held-Out Evaluation Window

In W2 (Table 3), `gorgo-static`—deploying the W1-learned weights without adjustment—achieves the best p50 (0.294 s) and p95 (1.510 s). `gorgo-hillclimb` achieves the best p99 (2.595 s); the margin over the next-best policy (`random`, 2.612 s) is 0.017 s, inside one standard error of the p99 estimator at this sample size, so the p99 win in W2 should be read as a tie rather than a reliable advantage—we quantify the noise floor in §6. The two GORGO variants together sweep all three percentiles; no single GORGO variant does. The p95 spread in W2 is 1.38×, wider than W1's 1.25×.

`gorgo-static` beats `gorgo-hillclimb` on p50/p95 in W2 because the W1-learned weights are near-optimal for the W2 distribution; continuing to tune on a similar distribution adds variance from each ES proposal. This is a known property of $(1{+}1)$-ES under stationary objectives. The practical takeaway is that operators can run `gorgo-hillclimb` during a tuning window and freeze the result into `gorgo-static` for production.

`gorgo-autotune` underperforms in W2. The median-of-rates fit inverts Equation 1 on trailing samples, but the rate $(\mathrm{TTFT} - \mathrm{rtt})/\mathrm{uncached}$ is unstable when uncached counts are small (high cache hit), inflating the estimate. `gorgo-hillclimb` avoids this because its objective integrates over the mixture rather than trying to disentangle it.

`simple-session-affinity`'s p95 inflates 40.4% from W1 to W2 and its p99 by 75.4%—a hash-to-replica mapping cannot rebalance when a hash bucket spikes in the second window. By contrast, `gorgo-static`'s W2 p95 (1.510 s) improves over its W1 p95 (1.605 s), a 6.0% gain; it is the only policy whose p95 improves across windows.

5.3 Stress at c=64

To stress-test the W2 result we double the in-flight-request cap from 32 to 64 and benchmark two further 30-minute windows: a nighttime window the day after W1 (Table 4), in the same time-of-day band, and a daytime window on W1 itself (Table 5). Both seed all GORGO variants with the W1-learned hyperparameters; `simple-session-affinity` did not complete the nighttime sweep and is omitted from that table.

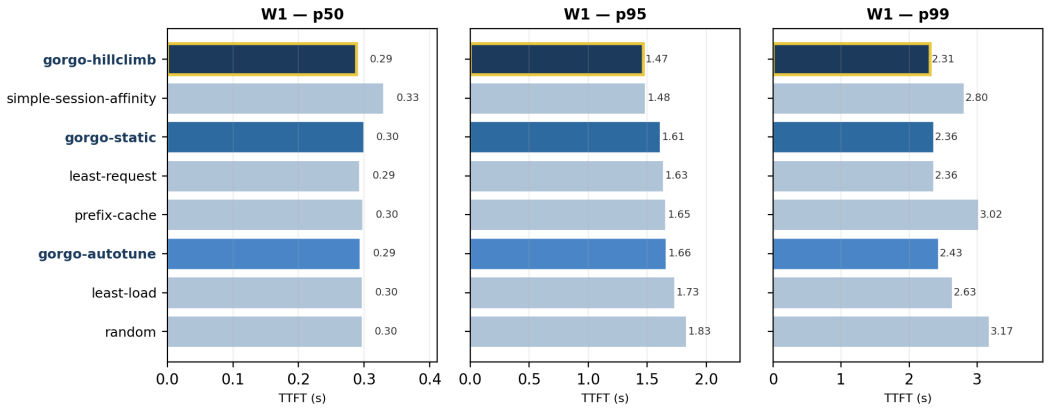

Figure 4: W1 TTFT broken out by percentile (sorted by p95, worst at top). `gorgo-hillclimb` (gold outline) wins all three percentiles during the tuning window.

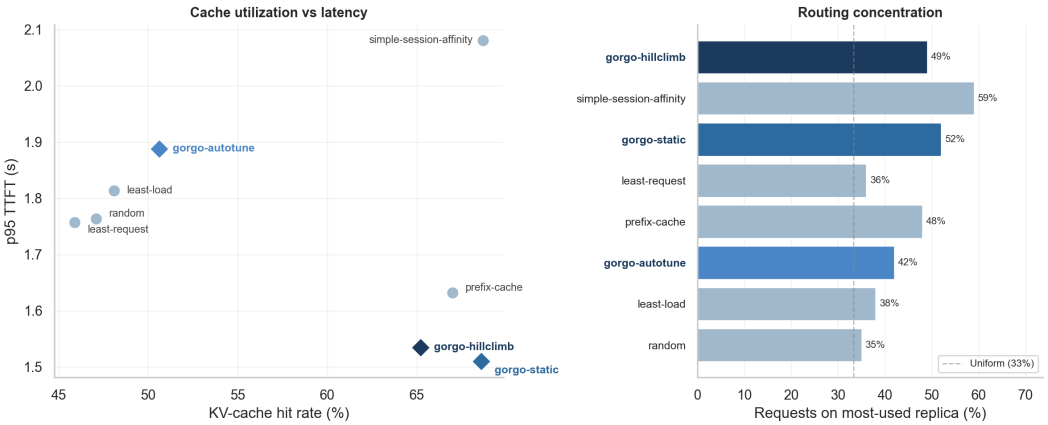

Figure 5: *Left*: KV-cache hit rate vs. p95 TTFT on W2. Bottom-right is ideal (high cache, low latency). `gorgo-static` and `gorgo-hillclimb` achieve both; `simple-session-affinity` achieves the highest cache hit rate but the worst p95 because it cannot rebalance load. *Right*: routing concentration (share of requests on the most-used replica). The dashed line marks uniform (33%). GORGO variants concentrate moderately on cache-warm replicas without the pathological imbalance of `simple-session-affinity`.

In the nighttime stress window (Table 4), `gorgo-static` wins p50 and `gorgo-hillclimb` wins p95 (1.331 s) and p99 (1.832 s); the two together sweep all three percentiles, and the hillclimb's p95 and p99 are 6.0% and 13.5% below its c=32 W2 result as the ES adapts to deeper queues.

In the daytime stress window (Table 5) the picture shifts. `gorgo-static` wins p50 (0.193 s) and p95 (1.400 s); `prefix-cache` wins p99 (1.973 s); `gorgo-hillclimb`'s p99 inflates to 7.186 s. Under midday traffic the live $(1{+}1)$-ES occasionally proposes weights that produce a heavy-tailed minute, and the negative-p95 objective does not penalize that excursion fast enough. The practical implication is the one W2 already suggested: freeze hillclimb-tuned weights into `gorgo-static` for production, and reserve `gorgo-hillclimb` for tuning windows where its exploration can be tolerated.

6 Limitations

Single trace, single seed. The percentile-sweep claim rests on two consecutive 30-minute windows of one GLM-5.1 trace, with one ES run per policy. W2 demonstrates that tuned weights survive a held-out window of the same workload; transfer across days, models, or arrival regimes is not characterized.

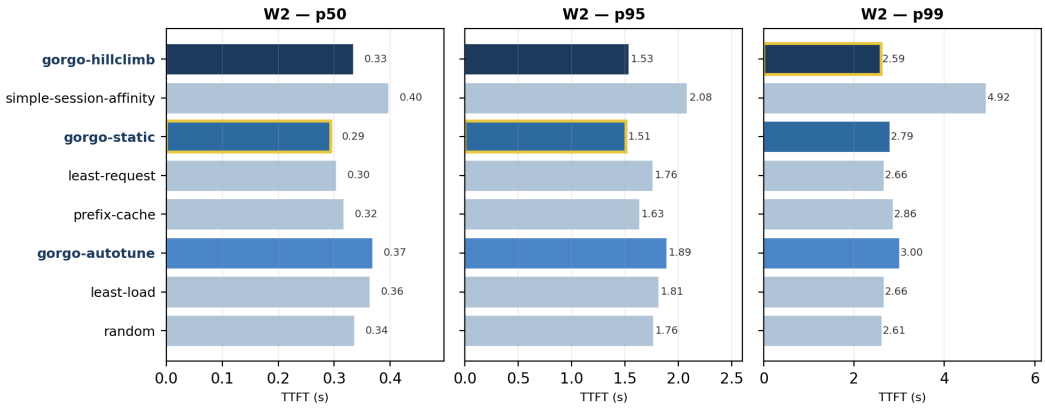

Figure 6: W2 TTFT broken out by percentile (sorted by W1 p95, worst at top). Gold outline marks the winner in each panel. `gorgo-static` (frozen W1 weights) wins p50 and p95; `gorgo-hillclimb` wins p99. `simple-session-affinity` degrades sharply at the tail (4.92 s p99).

p99 noise floor. Each window has $\approx 4{,}200$–$4{,}800$ successful responses, so the p99 standard error is $\sigma_{\mathrm{TTFT}}/\sqrt{n \cdot 0.01 \cdot 0.99} \approx 0.05$ s. The W2 p99 gap between `gorgo-hillclimb` (2.595 s) and `random` (2.612 s) is 0.017 s—inside one SE—and should be read as a tie; the W1 sweep and W2 p50/p95 wins are several SEs clear of zero.

Scope of fleet. Each policy runs on its own dedicated three-replica fleet of identical GPUs. Multi-tenant sharing, heterogeneous hardware, and prefill/decode disaggregation [Zhong et al., 2024, Patel et al., 2024] are not characterized here and would require additional cost terms. The score uses only HTTP-exposed metrics, not scheduler-internal signals [Pan et al., 2025, Yang et al., 2025a] that could tighten the prefill estimate.

Live-tuning instability. Running the $(1{+}1)$-ES live can produce a heavy-tailed p99 excursion (Table 5: `gorgo-hillclimb` p99 inflates to 7.186 s) because the negative-p95 objective does not penalize a single bad-minute proposal fast enough. The freeze-for-production recommendation in §5.3 is therefore a safety property, not just a convenience.

7 Related Work

Prefix caches and reuse. RadixAttention in SGLang [Zheng et al., 2024b] stores per-request KV state in a radix tree; PagedAttention in vLLM [Kwon et al., 2023] enables prefix sharing through paged memory. KVLink [Yang et al., 2025b], ChunkKV [Liu et al., 2025], KVFlow [Pan et al., 2025], and Learned Prefix Caching [Yang et al., 2025a] improve the single-replica cache but do not decide which replica serves a request.

Cross-replica routing and cross-region serving. Preble [Srivatsa et al., 2025] introduces longest-prefix-match routing across replicas with a load-balance fallback; AIBrix [AIBrix Team, 2024] packages a production-grade variant of the same design and supplies the baseline we run. Mooncake [Qin et al., 2024] is a KV-cache-centric architecture and the source of our trace format. SkyServe [Mao et al., 2025] and SkyWalker [Xia et al., 2025] address cross-region LLM serving at the placement and spillover layers respectively, but treat per-request routing as out of scope. k-LPM [Dexter et al., 2025] formulates LLM scheduling under TTFT constraints as NP-hard. DLPM [Cao et al., 2025] targets fairness with locality. Llumnix [Sun et al., 2024] migrates requests across replicas. Multi-LLM routers [Wu and Silwal, 2025] address model selection. CacheBlend [Yao et al., 2024] routes by trie-measured prefix length but does not measure network RTT. DistServe [Zhong et al., 2024] and Splitwise [Patel et al., 2024] disaggregate prefill from decode. GORGO is the first single-router policy in this space that scores cache locality, replica load, and wide-area latency in one cost and

derives the cost weights from the deployment's own per-request TTFT stream, with no engine modifications.

Online hyperparameter adaptation. Bayesian and bandit-based methods have been applied to serving configuration [Wu and Silwal, 2025]. For a 2-dimensional parameter space the $(1+1)$-ES [Rechenberg, 1973] provides fast, overhead-free convergence with no surrogate model.

8 Conclusion

Routing across LLM replicas is a three-signal decision—cache locality, replica load, and wide-area latency—but production heuristics commit to one signal and degrade whenever it stops being the binding constraint. We treat the three as terms of an additive per-replica cost and let online TTFT feedback set the two scaling weights, in place of operator-tuned constants or offline profiling. On a long-context, high-prefix-reuse production trace the resulting policy family collectively achieves the lowest TTFT at every reported percentile in both a tuning and a held-out evaluation window; on a short-prompt, low-reuse public trace (Appendix A) the same policy is non-competitive, in agreement with the regime characterization in §2. The advantage is therefore a property of the workload regime as much as of the policy: it scales with how much of TTFT is recoverable through prefill reuse, queue avoidance, or network selection rather than fixed by hardware. Two implementation choices made this practical without engine modifications. First, the cost-model weights are derived from the deployment's own per-request TTFT stream rather than from a calibration run, which removes a per-deploy operator step and makes the policy robust to heterogeneous fleets where each replica has its own per-token prefill rate. Second, the weights are frozen once they stabilize rather than re-tuned indefinitely; the best evaluation-window configuration in our experiments was the one that froze the tuning-window weights. Both choices transfer to deployments where the workload regime is not known in advance and a dedicated calibration window before each redeploy is not affordable.

References

AIBrix Team. AIBrix: An open-source infrastructure for LLM inference at scale. `https://github.com/vllm-project/aibrix`, 2024.

Shiyi Cao, Yichuan Wang, Ziming Mao, Pin-Lun Hsu, Liangsheng Yin, Tian Xia, Dacheng Li, Shu Liu, Yineng Zhang, Yang Zhou, Ying Sheng, Joseph Gonzalez, and Ion Stoica. Locality-aware fair scheduling in LLM serving. *arXiv preprint arXiv:2501.14312*, 2025.

Gregory Dexter, Shao Tang, Ata Fatahi, Qingquan Song, Tejas Dharamsi, and Aman Gupta. LLM query scheduling with prefix reuse and latency constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

GLM Team. GLM-4: A family of open multilingual multimodal chat LLMs, 2024.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Yue Liu, Bo Li, Xuming Hu, and Xiaowen Chu. ChunkKV: Semantic-preserving KV cache compression for efficient long-context LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Ziming Mao, Tian Xia, Zhanghao Wu, Wei-Lin Chiang, Tyler Griggs, Romil Bhardwaj, Zongheng Yang, Scott Shenker, and Ion Stoica. SkyServe: Serving ai models across regions and clouds with spot instances. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 2025. doi: 10.1145/3689031.3717459.

Zaifeng Pan, Ajjkumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2507.07400.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024.

Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yanzhe Wu, Weimin Zheng, Xinran Chen, Rui Xu, Xingda Yao, Rongxin Wei, Zhiyuan Zhao, Jing Huang, Mingjie Yang, and Jidong Wu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving, 2024.

Qwen Team. Qwen3 technical report. 2025. Alibaba Cloud.

Ingo Rechenberg. Evolutionsstrategie: Optimierung technischer systeme nach prinzipien der biologischen evolution. 1973.

Vikranth Srivatsa, Sumanth Madhavan, Tian Hu, Sarah Tarun, Yuhan Cheng, Wei Han, Jingyu Yu, Roberto Cristian Rusti, Lifeng Wang, Yiying Liu, and Hao Zhang. Preble: Efficient distributed prompt scheduling for LLM serving. In *International Conference on Learning Representations (ICLR)*, 2025.

Biao Sun, Ziming Huang, Hanyu Tang, Chengquan Wang, Cheng Zhang, Yiming Li, Liu Yang, Wencong Zhang, Lin-Wei Wang, Tianhe Wu, Ang Cao, Yongqiang Lin, et al. Llumnix: Dynamic scheduling for large language model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

Fangzhou Wu and Sandeep Silwal. Efficient training-free online routing for high-volume multi-LLM serving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Tian Xia, Ziming Mao, Jamison Kerney, Ethan J. Jackson, Zhifei Li, Jiarong Xing, Scott Shenker, and Ion Stoica. SkyWalker: A locality-aware cross-region load balancer for LLM inference. arXiv preprint arXiv:2505.24095, 2025.

Dongsheng Yang, Austin Li, Kai Li, and Wyatt Lloyd. Learned prefix caching for efficient LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025a.

Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. KVLink: Accelerating large language models via efficient KV cache reuse. *arXiv preprint arXiv:2502.16002*, 2025b.

Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Peng, Hao Zhang, Ion Stoica, Kurt Keutzer, and Michael W. Mahoney. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion, 2024.

Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. WildChat: 1M ChatGPT interaction logs in the wild. In *International Conference on Learning Representations (ICLR)*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. In *International Conference on Learning Representations (ICLR)*, 2024a.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024b.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

NeurIPS Paper Checklist

1. **Claims**
   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?
   Answer: [Yes]
   Justification: The abstract claims (i) public datasets underrepresent long-context multi-turn traffic (quantified in §2), (ii) we evaluate routing policies on a production trace (Tables 2, 3, 4, 5), and (iii) GORGO sweeps every TTFT percentile in both 30-minute windows. We are explicit in the introduction, §5.3, and §6 that "GORGO" refers to a family, that the W2 p99 margin is at the noise floor, and that on the daytime stress window `gorgo-hillclimb`'s p99 inflates and `prefix-cache` wins p99 instead.

2. **Limitations**
   Question: Does the paper discuss the limitations of the work?
   Answer: [Yes]
   Justification: Section 6 lists single-trace generalization, the W2 p99 noise floor, the single-tenant assumption, `gorgo-autotune` instability, hardware homogeneity, the absence of PD-disaggregation, and ES convergence time.

3. **Theory assumptions and proofs**
   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete proof?
   Answer: [N/A]
   Justification: The paper presents an additive cost model and an online optimizer with no formal theorems.

4. **Experimental result reproducibility**
   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?
   Answer: [Yes]
   Justification: Section 4 specifies model, regions, hardware, fleet topology, concurrency, trace windows, and per-window GORGO seeding. Section 3 gives Equation 2, the $(1+1)$-ES configuration ($\sigma_0 = 0.5$, 1/5-rule, hop 16, window 64), and the median-of-rates fit. The reproducibility statement enumerates four canonical paths in the released artifact.

5. **Open access to data and code**
   Question: Does the paper provide open access to the data and code?
   Answer: [Yes]
   Justification: The proxy and policy code is available at `https://anonymous.4open.science/r/GORGO-D25A/README.md`. The GLM-5.1 production trace is not redistributable under its terms of service. The public-dataset characterization (LMSYS, WildChat) is fully reproducible with no proprietary inputs.

6. **Experimental setting/details**
   Question: Does the paper specify all the training and test details?
   Answer: [Yes]
   Justification: Section 4 specifies model, hardware, regions, concurrency, trace windows, max input tokens, and per-window seeding for adaptive policies. ES configuration is given in Section 3.

7. **Experiment statistical significance**
   Question: Does the paper report error bars or appropriate statistical significance information?
   Answer: [Yes]
   Justification: Section 6 derives an approximate p99 standard error and shows that the W2 p99 margin between `gorgo-hillclimb` and `random` is inside one standard error, while the W1 sweep and W2 p50/p95 wins are several standard errors clear of zero.

8. **Experiments compute resources**
   Question: Does the paper provide sufficient information on compute resources?
   Answer: [Yes]
   Justification: Each policy runs on a dedicated 3-replica fleet of L40S:2 SGLang servers

(tensor-parallel 2, 96 GB VRAM per replica). Eight policies $\times$ two windows $=$ 16 fleet-runs, each $\sim$30 min wall clock, on three regions. Total GPU-hours: $\approx$24.

9. **Code of ethics**
   Question: Does the research conform with the NeurIPS Code of Ethics?
   Answer: [Yes]
   Justification: The work concerns routing infrastructure. The production trace is used under terms that permit research replay; we release only aggregated prefix-trie statistics, not individual requests. No individuals are identifiable from the released aggregates.

10. **Broader impacts**
    Question: Does the paper discuss potential societal impacts?
    Answer: [Yes]
    Justification: Routing improvements reduce the energy and dollar cost of serving LLMs at fixed quality. Lower-cost serving may accelerate deployment in settings where that is socially harmful; the contributions are infrastructure-level and do not affect model capability.

11. **Safeguards**
    Question: Does the paper describe safeguards for responsible release?
    Answer: [N/A]
    Justification: We do not release datasets or models. The released code is a routing proxy and policy library with no inherent dual-use beyond LLM-serving infrastructure.

12. **Licenses**
    Question: Are creators or owners of assets properly credited?
    Answer: [Yes]
    Justification: SGLang, vLLM, AIBrix, LMSYS-Chat-1M, WildChat-4.8M, and Qwen3 are cited and used under their published licenses.

13. **New assets**
    Question: Are new assets introduced in the paper well documented?
    Answer: [Yes]
    Justification: The proxy, policy library, experiment controller, and trace builder are documented in the repository's top-level documentation. Spec and manifest files are stored alongside the controller.

14. **Crowdsourcing and research with human subjects**
    Question: Does the paper involve crowdsourcing or human subjects?
    Answer: [N/A]
    Justification: No human subjects.

15. **IRB approvals**
    Answer: [N/A]
    Justification: No human subjects.

16. **Declaration of LLM usage**
    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the research methodology?
    Answer: [N/A]
    Justification: LLM serving infrastructure is the object of study. Authors used LLMs for routine writing assistance only.

Reproducibility Statement

The experiments are driven by a single experiment controller that launches fleets, configures policies, and replays a trace per policy. The four canonical paths in the released artifact are:

- `specs/policy_matrix_abstract_night.json` — W1 spec (eight policies, manual GORGO starter weights);
- `specs/policy_matrix_abstract_night_w2.json` — W2 spec (eight policies, GORGO seeded with W1-learned weights);
- `experiment_runner/policy_matrix_app.py` — experiment controller;
- `data_processing/build_mooncake_trace.py` — trace builder.

W1 and W2 manifests, result CSVs, and summary plots are produced by running the controller against these specs and post-processing per-policy stats with the released analysis scripts. The GLM-5.1 production trace is not redistributable; the public-dataset characterization (LMSYS, WildChat) reproduces with no proprietary inputs.

A WildChat replay (regime-sensitivity control)

We replay a 30-minute window of WildChat-4.8M [Zhao et al., 2024] on the same 2×L40S three-region fleet, with the same eight policies as the GLM-5.1 sweeps. WildChat averages 2,925 input tokens per request and 5.3% intra-user prefix reuse—an order of magnitude shorter than GLM-5.1 and roughly a tenth the reuse—putting it well outside the regime characterized in §2.

Table 6: WildChat-4.8M replay: 30-minute window, c=32. TTFT in seconds. Bold is best in column. Compared to the GLM-5.1 results, GORGO variants are not competitive: `gorgo-static` and `gorgo-autotune` are the worst two policies on p95.

Policy	Completed	p50	p95	p99	max	Succ.%
simple-session-affinity	20,447	0.416	**1.025**	**1.712**	7.75	99.6
least-request	20,517	0.480	1.036	1.731	110.45	100.0
random	20,436	**0.416**	1.077	1.796	15.82	99.6
prefix-cache	20,504	0.497	1.186	2.588	10.63	99.9
least-load	20,484	0.532	1.217	2.535	8.28	99.8
gorgo-hillclimb	20,473	0.541	1.325	2.654	7.05	99.8
gorgo-static	20,462	0.709	1.932	3.969	60.18	99.7
gorgo-autotune	20,451	0.541	3.583	6.438	12.55	99.7

The ranking is reversed: `simple-session-affinity` wins p95 and p99 by a large margin, `least-request` is competitive, `gorgo-hillclimb` sits mid-pack, and `gorgo-static` and `gorgo-autotune` are the worst two policies. Two mechanisms drive this. First, the prefill term in Equation 2 is small at 2,925 tokens; the cost-model weights are mostly fitting noise. Second, intra-user reuse is 5.3% rather than 53%; even a perfect cache-routing decision saves only a few hundred tokens per request, well inside the cross-region RTT band, so chasing cache locality is counter-productive when paired with a cross-region routing penalty. The `gorgo-static` weights frozen from a GLM-5.1 W1 are particularly mis-calibrated to this regime—a reminder that the cost-model parameters do not transfer across workload classes.

We include this result as a regime-sensitivity control, not as a contribution. The takeaway is the same one stated in §2: the gain from joint cache–network–queue routing depends on the workload having long prompts and substantial intra-user prefix reuse. Public chat datasets do not exercise this regime.

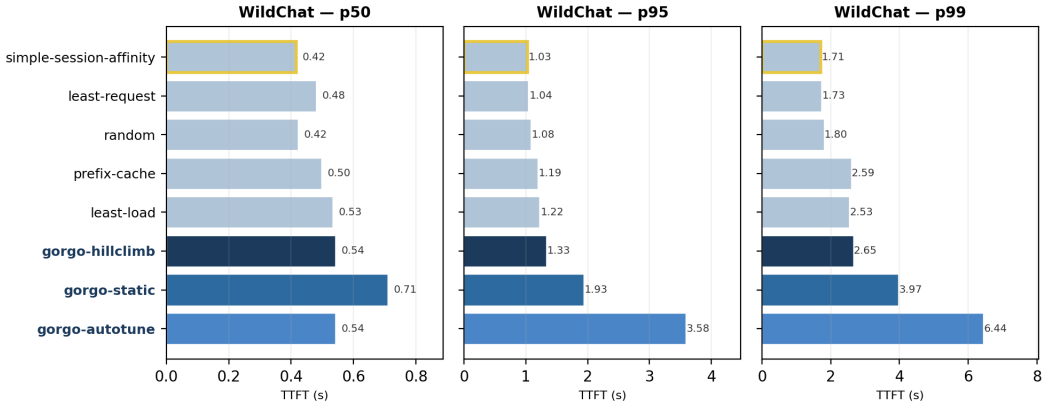

Figure 7: WildChat-4.8M replay, per-percentile TTFT (p50/p95/p99) across the eight policies. GORGO variants (dark blues) are not competitive: `gorgo-static` and `gorgo-autotune` are the slowest two policies at the tail, and `gorgo-hillclimb` sits mid-pack. The ranking inverts the GLM-5.1 result, consistent with the regime argument in §2.